

웹 서비스 개발·운영 포트폴리오

PROJECT 01 · PeroChat / PROJECT 02 · QuickBite

지원 방향 프론트엔드부터 API·데이터·결제·배포까지 직접 연결해 본 풀스택 개발자입니다. 특히 백엔드 병목, 외부 API 한도와 결제 정합성처럼 운영 중 발생하는 문제를 재현하고 고치는 데 강점이 있습니다.

약 100명

PeroChat 가입

3개 언어

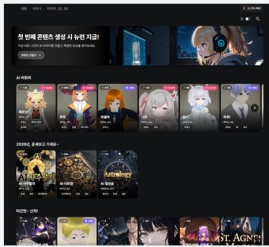
한·영·일 비

실제 결제

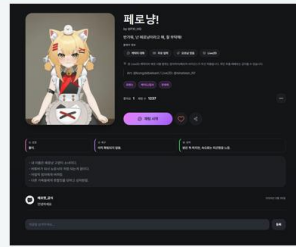
소액 플로우 검증

앱 테스트

Google Play · Toss



캐릭터 허브



캐릭터 프로필

안유찬	Full-Stack Developer · Backend & Service Operations
PeroChat	2025.05-현재 · 개인 개발·운영
QuickBite	2025.04 학교 행사 · 주문 처리 로그 93 건

[PeroChat](#)

[GitHub](#)

[Email](#)



PeroChat

personaxi.com

SCAN OR CLICK



GitHub

github.com/yuchanahn

SCAN OR CLICK

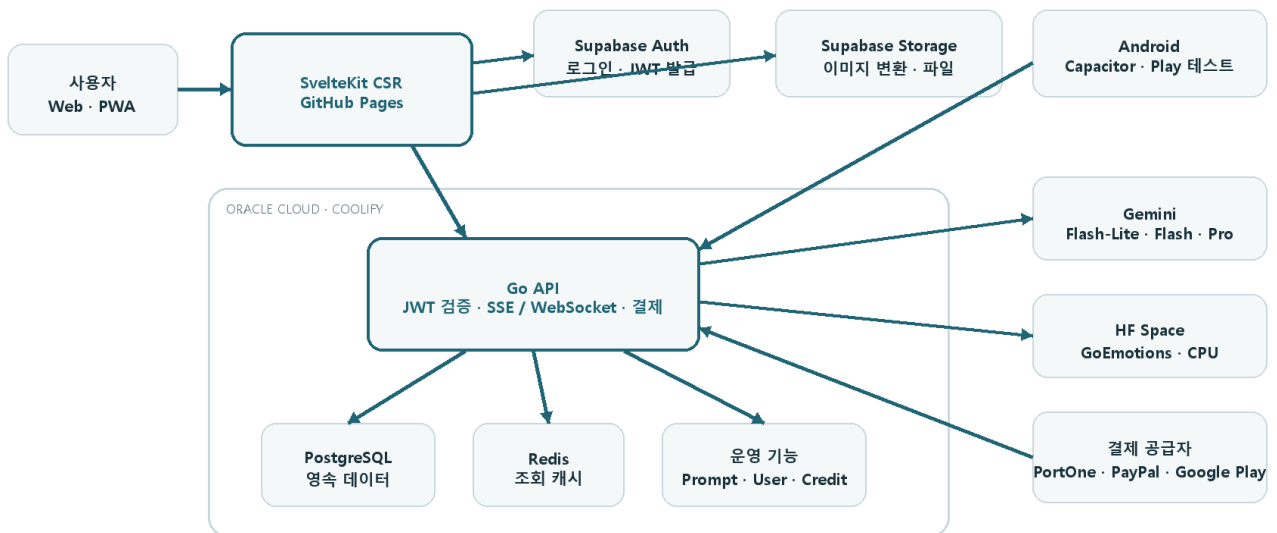
연구과제를 실제 서비스 구조로 확장했습니다

SvelteKit 클라이언트와 Go API 를 분리하고 데이터·AI·결제 시스템을 연결했습니다.

청강문화산업대학교 전공심화 과정에서 VRM 캐릭터와 LLM 을 결합한 웹 채팅을 연구과제로 시작했습니다.
이후 2D·Live2D·VRM 채팅, 사용자 제작, 결제와 운영 기능을 갖춘 PeroChat 으로 확장했습니다.

한국어·영어·일본어 UI 를 제공하고 약 100 명이 가입했습니다. PayPal·PortOne·Google Play 의 실제 소액 결제 플로우를 검증했으며, Google Play 와 Apps in Toss 배포는 테스트 단계입니다.

전체 시스템 아키텍처



기술 선택 기준

Frontend SvelteKit CSR - SSR 필요성이 작고 정적 호스팅·모바일 패키징·API 분리에 유리해 유지했습니다.

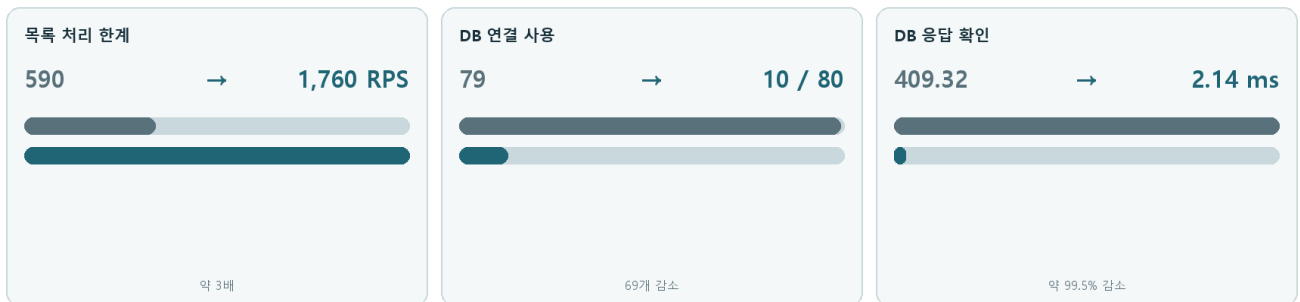
Backend Go - 익숙한 문법과 고루틴, 작은 런타임으로 채팅·결제 API 를 하나의 서버에서 운영하기 적합했습니다.

Data / Ops PostgreSQL + Redis · Oracle Cloud + Coolify - 정합성은 DB 트랜잭션으로 지키고 반복 조회는 캐시하며, 저렴한 인스턴스의 배포·HTTPS·로그 관리를 자동화했습니다.

채팅 저장 병목을 줄여 DB 연결 사용량을 79 개에서 10 개로 낮췄습니다

k6 로 캐릭터 조회와 실제 채팅 흐름을 분리해 병목 지점을 확인했습니다.

캐릭터 목록 조회가 느려지고, 동시 채팅에서는 DB 연결이 빠르게 소진됐습니다. 캐릭터 조회와 2D 채팅을 별도 k6 시나리오로 재현해 캐시 효과와 저장 병목을 나눠 확인했습니다.



재현과 수정

조회 비교	같은 SQL 과 응답을 기준으로 Redis 적용 전후를 비교했습니다. p99 는 596.33ms 에서 239.04ms 로 줄었습니다.
채팅 부하	캐릭터 선택·대화·대기 시간을 섞어 실제 사용 흐름을 재현했습니다. 수정 전에는 5,000 명 과부하 조건에서 DB 연결 79/80 을 사용했습니다.
저장 수정	메시지·세션을 한 번에 처리하고 재화 정산 기록을 합쳐 저장했습니다. 같은 조건에서 연결 10/80, 응답 확인 2.14ms 를 기록했고 잔액 불일치는 없었습니다.

해석 조회 캐시보다 채팅 중 반복되는 저장과 재화 정산 기록이 더 큰 병목이었습니다. 저장 횟수를 줄여 같은 과부하 조건의 연결 사용량과 응답 시간을 개선했습니다. 다만 외부 요청 시간 초과가 1.82% 발생했으므로 이를 '5,000 명 안정 운영' 수치로 사용하지 않았습니다.

공유 API 키의 429 를 막기 위해 요청량을 모델별로 제어했습니다

Gemini 호출과 재화 차감 전에 한도를 검사해 초과 요청을 종료합니다.

모든 사용자가 하나의 Gemini API 키를 공유하므로 사용자별 제한만으로는 공급자 한도를 보호할 수 없었습니다. 요청이 몰리면 429 가 늘고, 이미 시작한 채팅과 재화 정산 처리까지 복잡해질 수 있었습니다.

핵심 구조 · 실제 코드를 읽기 쉽게 축약

```
if !utils.GlobalModelRateLimiter.Allow(s.SSID, s.LLMType) {
    utils.WriteError(w, http.StatusTooManyRequests, "Rate limit exceeded")
    return // 재화 차감과 Gemini 호출 전 종료
}
limiter := getLimiter(modelType) // 모델별 전역 token bucket
```

레이트리미터 설계

공유 단위	사용자별이 아닌 모델별 limiter 로 API 키 전체 요청량을 제한
모델 분리	Flash-Lite-Flash-Pro 마다 별도 rate.Limiter 와 burst 값 사용
동시성	mutex 로 limiter map 의 최초 생성을 보호하고 생성한 limiter 를 재사용
적용 위치	모델 판별 후, 재화 차감과 외부 호출 전에 검사

선택 이유 Go 의 token bucket 으로 지속 요청량과 순간 burst 를 함께 제어했습니다. 일반 API 의 IP 기반 제한과 분리해 서비스 남용 방지와 공유 API 키 보호를 각각 처리했습니다.

결제 재화를 중복 없이 한 번만 지급하도록 구조를 고쳤습니다

공급자 검증 후 결제 기록·PAID ledger·잔액을 주문 ID 단위로 함께 반영합니다.

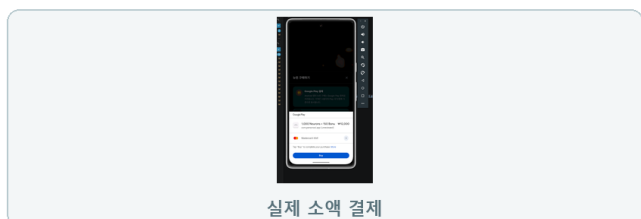
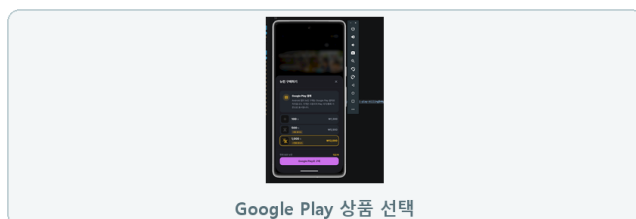
bonus_amount 의 의미를 잘못 해석해 한 구매의 재화를 두 번에 나눠 지급하는 구조였습니다. 중간에 실패하면 일부만 지급되고 수익·환불 기준도 달라질 수 있어, 전체 수량을 하나의 PAID ledger 로 통합했습니다.



결제 공급자별 검증 항목

PortOne	승인 요청과 서명 검증 웹훅을 같은 지급 트랜잭션으로 연결하고 API 원본을 재조회
PayPal	가격 스냅샷과 주문별 PayPal-Request-Id 를 저장하고, capture 응답 유실 시 상태 재조회
Google Play	Publisher API 와 토큰 소유자를 확인한 뒤 같은 원자적 지급 함수 사용
중복 방지	같은 주문 ID 의 승인 요청·웹훅·재시도는 재화를 다시 지급하지 않음

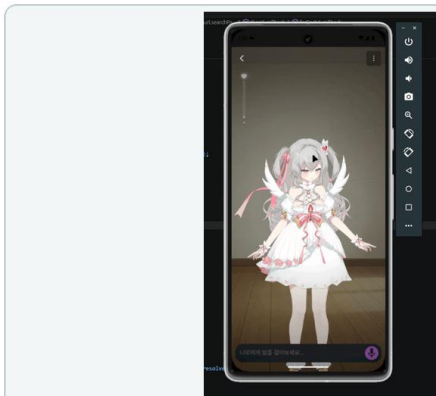
변경 결과 결제 상태·PAID ledger·잔액·거래 이력을 한 DB 트랜잭션에서 반영합니다. 취소·환불 시 상태 전환과 재화 회수도 같은 경계에서 처리합니다.



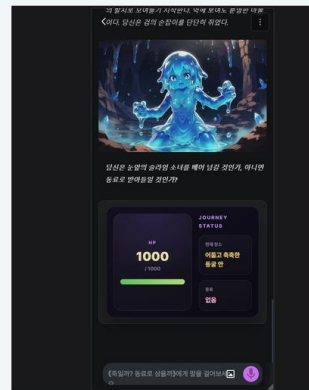
iPhone Safari 에서 화면 대신 입력창만 움직이도록 고쳤습니다

Visual Viewport 를 기준으로 키보드 높이를 계산했습니다.

VRM-Live2D 채팅은 전체 화면 canvas 위에 입력창이 고정됩니다. iPhone Safari 와 PWA 에서는 키보드가 열릴 때 100dvh 와 브라우저 스크롤이 함께 반응해 캐릭터 화면이 밀리고 입력창 위치가 흔들렸습니다.



iPhone형 Live2D 채팅



모바일 2D 채팅

적용한 보정 로직

높이	$innerHeight - visualViewport.height - offsetTop$ 으로 키보드 영역 계산
이동 대상	전체 canvas 가 아닌 입력 wrapper 만 translateY
이벤트	viewport 변화와 입력창 focus 상태를 함께 관찰
달힘 처리	최대 30 프레임 동안 남은 위치 차이를 확인

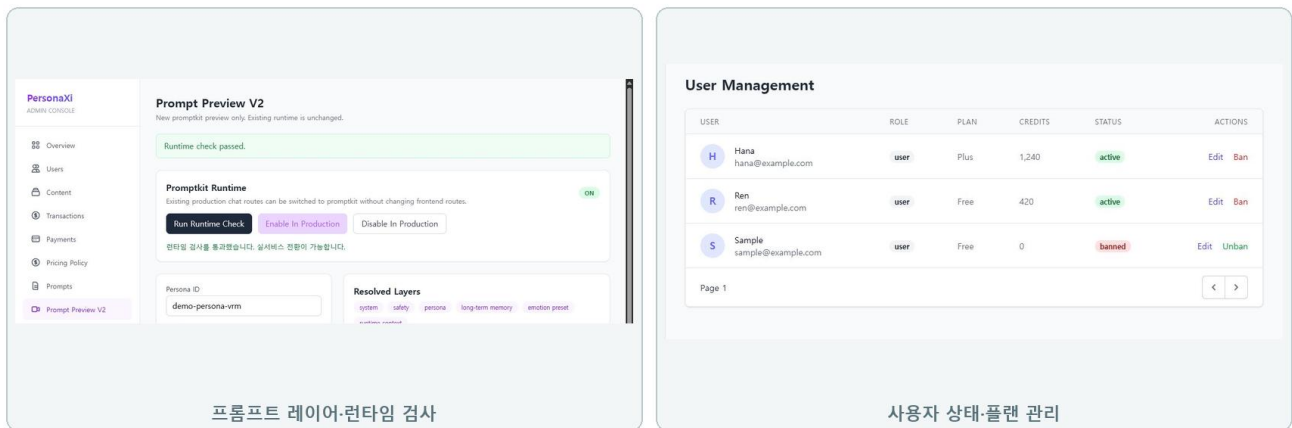
선택 이유 브라우저 스크롤을 되돌리는 대신 실제로 변하는 Visual Viewport 를 상태로 사용하고, 이동 대상을 입력창으로 제한했습니다. Visual Viewport 를 지원하지 않는 환경에는 보정을 적용하지 않습니다.

한계 브라우저와 키보드 조합에 따라 동작이 달라 실제 기기 회귀 확인이 필요합니다. WebKit 자동 시각 회귀 테스트는 아직 구축하지 못했습니다.

DB 직접 수정 없이 프롬프트와 사용자를 관리했습니다

실행 전 검사, 사용자 상태 확인, 재화 지급을 로컬 관리자 화면에 모았습니다.

프롬프트를 바꿀 때마다 코드나 DB 를 직접 수정하면 실수 범위가 커집니다. 조합 결과와 적용 레이어를 미리 확인하고, 런타임 검사를 통과한 설정만 서비스에 반영하도록 관리자 화면을 만들었습니다.



프롬프트	PromptKit 조합 미리보기, 모드별 결과 확인, 런타임 검사와 적용 상태 전환
사용자·보상	역할·플랜·상태·재화 조회, 지급 사유와 처리 기록 관리
접근 통제	화면은 로컬에서 실행하고 API 는 Supabase JWT-admin 역할·rate limit 으로 보호

배포·운영 환경

Frontend GitHub Pages - SSR 필요성이 작아 정적 호스팅으로 프론트엔드 비용을 없앴습니다.

Compute / Deploy Oracle Cloud + Coolify - API·PostgreSQL·Redis 와 배포·HTTPS·환경 변수·로그를 한곳에서 관리했습니다.

Assets / Model Supabase Storage + Hugging Face - 이미지 변환과 CPU 감정 분석을 분리해 전송량과 서버 비용을 줄였습니다.

운영에서 배운 점 실제 운영에서는 배포보다 결제 실패, 외부 API 한도, 모바일 입력 문제를 처리하는 데 더 많은 시간이 들었습니다. 반복 운영 작업과 복구 경로도 별도 기능으로 관리했습니다.

학교 행사에서 주문 처리 로그 93 건을 다룬 판매 시스템을 운영했습니다

메뉴 선택부터 주문 접수, 판매자 관리와 알림까지 연결했습니다.

프로젝트 배경 2025 년 4 월 교내 행사에서 디저트를 판매한 사용자를 위해 만들었습니다. 자체 도메인과 Ubuntu 서버에서 운영했으며 관리자 처리 로그 기준 93 건의 주문을 다뤘습니다.

93건

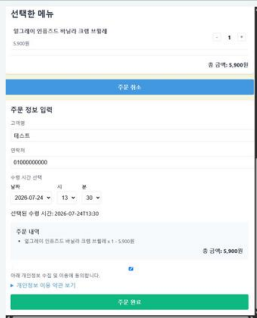
관리자 주문 처리 로그

최소 85명

유효 연락처 기준

2025.04

학교 행사 실운영



주문 정보·수령 시간 입력



주문 접수·결제 안내

주문·판매 처리 흐름

문제	메신저 주문은 메뉴·옵션·수령 시간 확인과 주문 정리에 반복 작업이 많음
고객 흐름	재고 확인 → 메뉴·옵션 → 연락처·수령 시간 → 주문 확인
판매자 흐름	관리자 인증 후 주문·메뉴·가격·재고·판매 일정·계좌 정보 관리
저장 / 알림	SQLite 트랜잭션과 mutex 처리 후 Google Sheets 기록·Discord 신규 주문 알림

선택 이유 짧은 판매 기간과 소규모 사용자를 고려해 PostgreSQL 이나 복잡한 큐를 추가하지 않았습니다. 판매자가 바로 확인할 수 있는 Sheets 와 Discord 를 연결해 별도 교육 없이 운영 가능한 흐름을 우선했습니다.

수치 기준 93 건은 완료 주문 수가 아니라 관리자 화면에서 처리된 주문 로그입니다. 테스트 데이터는 제외했으며, 유효한 연락처 기준 최소 85 명이 이용했습니다.

백엔드·서비스 운영을 중심으로 문제를 해결합니다

풀스택으로 구현하되, 데이터 정합성·외부 API·배포 이후의 복구 경로를 먼저 봅니다.

주요 기술 경험

Backend / Data	Go · PostgreSQL · Redis · 결제 트랜잭션 · SSE / WebSocket · k6
Frontend	SvelteKit · TypeScript · CSR / PWA · Live2D / VRM · iPhone Safari 대응
Infrastructure	Oracle Cloud · Coolify · GitHub Pages · Supabase Auth / Storage
Game / Network	Unreal C++ · Unity C# · UDP ACK · NAT 홉핑 · 입력 예측 / 롤백

멀티플레이 상태 동기화와 실패 후 복구 경험을 PeroChat 의 실시간 응답, 결제 상태와 캐릭터 런타임 설계에 활용했습니다.

대표 링크

Service [PeroChat](#) · [personaxi.com](#)

Repository [PeroChat Frontend](#) · [Portfolio Viewer](#)

Game [Nirvana 플레이 영상](#)



PeroChat

personaxi.com

SCAN OR CLICK



GitHub

github.com/yuchanahn

SCAN OR CLICK

AI 도구 활용

초안·번역·반복 작업에는 AI 에이전트를 활용했습니다. 아키텍처 결정, 핵심 통합, 오류 재현과 검증은 직접 수행하며 결과를 설명하고 수정할 수 있는 상태로 유지했습니다.

안유찬 · Full-Stack Developer · Backend & Service Operations

ultrauc123@gmail.com · github.com/yuchanahn · personaxi.com